

TokenNet

Virtual Distributed Network Token Ring System

Problem Statement No. 44

Data Structures and Algorithms

Course Project Documentation

1. Student details

Field	Details
Name	Aryaa Bhadane
Roll No	150096725057
Cohort	Sam Altman

2. Project title

TokenNet: Virtual Distributed Network Token Ring System

A C++ implementation of a secure corporate communication network that manages token-based access, router configurations, request processing, link quality routing, and bandwidth allocation using data structures and algorithms.

3. Problem statement

TokenNet models a secure corporate network where each computer needs a digital permission token before it can send data. The system we started with had six concrete problems that needed fixing:

- Router configuration lookups were slow and returned inconsistent results, because configs were stored in a flat, unsorted structure.
- If someone accidentally revoked a security token, there was no way to undo it. The action was permanent.
- Computers requesting network access were handled in no particular order, so a machine that asked first could end up waiting behind one that asked later.
- The system had no way to rank links by quality (speed, reliability, error rate). All links were treated as equal.
- There was no model of the network's physical topology. Without one, you cannot compute cost-efficient routes between offices.
- Bandwidth was handed out on a first-come basis instead of by priority, which meant a file backup could starve a live VoIP call.

This project builds a replacement system in C++ that addresses each of these problems with a targeted data structure.

4. Objectives

- 1. Use a balanced BST (`std::map`) so router configurations can be inserted, deleted, and looked up in $O(\log n)$ time.
- 2. Add a stack-based undo mechanism that lets operators reverse the last token grant or revoke instantly.
- 3. Process network access requests in strict FIFO order using a queue.
- 4. Support channel lookup over unsorted connection lists using linear search.
- 5. Sort network links by quality score (descending) with merge sort, giving $O(n \log n)$ worst-case performance.
- 6. Model the corporate network as a weighted graph, stored as an adjacency list.
- 7. Compute minimum-cost routing paths using Dijkstra's algorithm on that graph.
- 8. Allocate bandwidth to the highest-priority data stream first, using a max-heap priority queue.

5. Design and architecture

The system is split into eight independent C++ classes, each responsible for one concern. They share no state. You can test, modify, or replace any module without touching the others.

System architecture

```
TokenNet System
|
+-- RouterSettings      -> std::map (Balanced BST)
+-- PermissionUndo      -> std::stack (LIFO)
+-- RequestLine         -> std::queue (FIFO)
+-- ChannelLookup       -> Linear Search on std::vector
+-- QualitySorter       -> Merge Sort on vector<Link>
+-- CompanyNetwork      -> Graph (adjacency list)
|   +-- cheapestRoute() -> Dijkstra (min-heap)
+-- TrafficController   -> std::priority_queue (max-heap)
```

Mapping to DSA course modules

Feature	Data Structure	Algorithm	Course Module
Router Settings	std::map (BST)	Insert/Search/Del	Mod 4 - Trees
Permission Undo	std::stack	Push/Pop (LIFO)	Mod 3 - Stacks
Request Line	std::queue	Enqueue/Dequeue	Mod 4 - Queues
Channel Lookup	std::vector	Linear Search	Mod 2 - Searching
Quality Sorter	vector<Link>	Merge Sort	Mod 2 - Sorting
Company Network	Adjacency List	Graph Build	Mod 5 - Graphs
Cheapest Route	Min-Heap PQ	Dijkstra	Mod 5 - Graph Apps
Traffic Control	Max-Heap PQ	Priority Sched.	Mod 4 - Heaps

Each class is self-contained. The main() function at the bottom of the source file acts as a demo driver: it creates instances, feeds them sample data, and prints the results to the console.

6. Algorithm description

1. RouterSettings (balanced BST via std::map)

std::map is backed by a red-black tree, which is a self-balancing binary search tree. Every insertion, deletion, and lookup takes $O(\log n)$ time because the tree rebalances itself after each mutation. We use the router ID (an integer) as the key and the configuration string as the value.

When displayAll() iterates the map, it walks the tree in-order, so routers come out sorted by ID automatically.

2. PermissionUndo (stack)

Every grant or revoke is pushed onto a stack as a pair<string, bool>. The string holds the computer name; the boolean records whether the action was a grant (true) or a revoke (false). Calling undoLast() pops the top element and reports which action was reversed. Both push and pop run in $O(1)$.

This mirrors how Cisco IOS handles configuration rollback: each change is logged sequentially, and the most recent one can be unwound instantly.

3. RequestLine (queue)

A standard FIFO queue. Computers call addRequest() to join the back of the line, and processNext() serves whoever is at the front. This guarantees that no machine gets skipped. The 802.1X port-based access control standard uses the same queuing pattern.

4. ChannelLookup (linear search)

The channel list is stored in an unsorted vector. findChannel() walks the vector from index 0 to the end, comparing each entry with the target string. If it finds a match, it returns the index; otherwise it returns -1. This is $O(n)$ in the worst case, which is fine when the list is short or unsorted.

5. QualitySorter (merge sort)

Merge sort divides the link array in half recursively until each piece has one element, then merges pieces back together in sorted order. We compare by quality score with $\geq$ so the result is descending (best links first). Merge sort runs in $O(n \log n)$ in all cases and is stable, meaning links with identical scores keep their original relative order.

6 & 7. CompanyNetwork + Dijkstra's algorithm

The network is stored as an adjacency list: an `unordered_map` where each key is a device name and the value is a vector of (neighbor, cost) pairs. `addLink()` adds edges in both directions because the network is undirected.

`cheapestRoute()` runs Dijkstra's algorithm starting from a source node. It initializes every node's distance to `INT_MAX` (infinity), sets the source to 0, and then repeatedly extracts the closest unprocessed node from a min-heap priority queue. For each neighbor, if the path through the current node is cheaper than the neighbor's recorded distance, it updates the distance and pushes the neighbor back into the heap. This is the standard textbook version with a lazy deletion approach (stale entries are skipped via the $d > \text{dist}[u]$ check). Total time: $O((V + E) \log V)$.

8. TrafficController (max-heap)

`std::priority_queue` is a max-heap by default. Each stream is stored as a (priority, name) pair, so the stream with the highest priority number sits at the top. `allocateBandwidth()` pops that top entry and assigns bandwidth to it. Real-world QoS schedulers (like those in enterprise routers) work the same way: VoIP traffic gets a higher priority class than bulk file transfers.

7. Complexity analysis

Feature	Data Structure	Time	Space	Stable?
Router Settings	std::map (BST)	$O(\log n)$	$O(n)$	N/A
Permission Undo	std::stack	$O(1)$	$O(n)$	N/A
Request Line	std::queue	$O(1)$	$O(n)$	N/A
Channel Lookup	Linear Search	$O(n)$	$O(1)$	N/A
Quality Sorter	Merge Sort	$O(n \log n)$	$O(n)$	Yes
Network Graph	Adjacency List	$O(V + E)$	$O(V + E)$	N/A
Cheapest Route	Dijkstra + Heap	$O((V+E)\log V)$	$O(V)$	N/A
Traffic Control	Max-Heap PQ	$O(\log n)$	$O(n)$	N/A

A few notes on these numbers:

- The $O(\log n)$ operations on std::map come from the red-black tree's guaranteed height of $O(\log n)$. Worst-case and average-case are the same.
- Merge sort's $O(n)$ extra space comes from the temporary left and right vectors created during each merge step.
- Dijkstra's $O((V + E) \log V)$ assumes a binary heap. Using a Fibonacci heap would improve this to $O(V \log V + E)$, but the STL does not provide one, and the constant factors rarely make it worthwhile for networks this size.

8. Screenshots of execution

Below is the terminal screenshot showing compilation and execution of the TokenNet program on macOS using g++.

Terminal output

```

● aryaabhadane@aryaas-MacBook-Air TokenNet % g++ TokenNet.cpp -o TokenNet
● aryaabhadane@aryaas-MacBook-Air TokenNet % ./TokenNet
=== TokenNet System Demo ===

-- Router Settings --
Router 101: Core-Router: 10.0.0.1
Router 202: Edge-Router: 192.168.1.1
Router 303: Backup-Router: 172.16.0.1
Search 202: Edge-Router: 192.168.1.1

-- Permission Undo --
Token GRANTED to PC-Alpha
Token GRANTED to PC-Beta
Token REVOKED from PC-Alpha
Undone: revoke for PC-Alpha

-- Request Line --
PC-Gamma added to request line.
PC-Delta added to request line.
Processing: PC-Gamma
Remaining: 1

-- Channel Lookup --
CH-3 found at index: 1

-- Quality Sorter --
Link-B: quality=90
Link-C: quality=75
Link-A: quality=60

-- Company Network + Cheapest Route --
HQ -> Branch-A: cost 4
HQ -> Branch-B: cost 6
HQ -> HQ: cost 0

-- Traffic Controller --
Stream 'File-Backup' added with priority 2
Stream 'VoIP-Call' added with priority 9
Stream 'Video-Conf' added with priority 7
Bandwidth allocated to: 'VoIP-Call' (priority 9)
Bandwidth allocated to: 'Video-Conf' (priority 7)

```

The output confirms that every module produces the expected results. The Dijkstra section is worth calling out: HQ to Branch-B costs 6 (routed through Branch-A at 4 + 2), cheaper than the direct link at cost 8. The algorithm correctly finds this indirect path.

9. Results and findings

Each module was tested by feeding it sample data in `main()` and comparing the output to hand-calculated expected values. Here is what we confirmed:

Router Settings

The balanced BST delivers consistent $O(\log n)$ lookups. Inserting three routers (IDs 101, 202, 303) and querying ID 202 returns the correct config string instantly. `displayAll()` prints them in ascending ID order without a separate sort step.

Permission Undo

After granting tokens to PC-Alpha and PC-Beta, then revoking PC-Alpha's token, calling `undoLast()` correctly identifies the revoke as the most recent action and reports it reversed. The stack handles it in $O(1)$.

Request Line

PC-Gamma enters the queue before PC-Delta. `processNext()` serves PC-Gamma first, and `pendingCount()` correctly reports 1 remaining. No request is skipped or reordered.

Channel Lookup

CH-3 is the second element (index 1) in the vector [CH-7, CH-3, CH-11]. Linear search finds it after two comparisons and returns 1.

Quality Sorter

Three links (quality 60, 90, 75) are sorted into descending order: 90, 75, 60. Merge sort handles this in $O(n \log n)$ time with stable ordering preserved.

Dijkstra's Cheapest Route

From HQ, the direct link to Branch-B costs 8, but the path through Branch-A costs $4 + 2 = 6$. Dijkstra correctly returns 6 for Branch-B and 4 for Branch-A. This matches the hand-calculated shortest paths.

Traffic Controller

VoIP-Call (priority 9) is allocated bandwidth before Video-Conf (priority 7), which comes before File-Backup (priority 2). The max-heap correctly orders streams by descending priority.

10. Conclusion

This project implements all eight required features of the TokenNet system. Each data structure was chosen to match the specific problem it solves:

- A balanced BST for $O(\log n)$ router lookups, because the alternative (a flat array or linked list) would degrade to $O(n)$.
- A stack for undo, because undo is inherently a last-in-first-out operation.
- A queue for request processing, because fairness requires first-in-first-out ordering.
- Linear search for channel lookup, because the data is unsorted and the list is small enough that $O(n)$ is acceptable.
- Merge sort for quality ranking, because it guarantees $O(n \log n)$ in the worst case and preserves the relative order of equal-quality links.
- An adjacency-list graph with Dijkstra's algorithm for routing, because this is the standard approach used by real routing protocols like OSPF.
- A max-heap for bandwidth allocation, because the scheduler needs to extract the highest-priority stream in $O(\log n)$ time.

These choices are not arbitrary. OSPF routers run Dijkstra on their link-state databases. Cisco IOS stores rollback checkpoints in a stack-like structure. IEEE 802.1X authenticators queue supplicants in FIFO order. The data structures in this project map directly to patterns used in production networking gear.

The modular, class-based design means each component can be tested on its own. If the network grows and linear search becomes too slow for channel lookup, you can swap in a hash set without changing anything else. The code compiles with standard C++17 and has no external dependencies.

11. GitHub repository link

[Please insert your GitHub repository URL here before submission]

[https://github.com/\[your-username\]/TokenNet](https://github.com/[your-username]/TokenNet)